LAB 7: Fuzzing for Input Validation Bugs

Automated Vulnerability Detection & Defensive Hardening using Hypothesis and Pytest

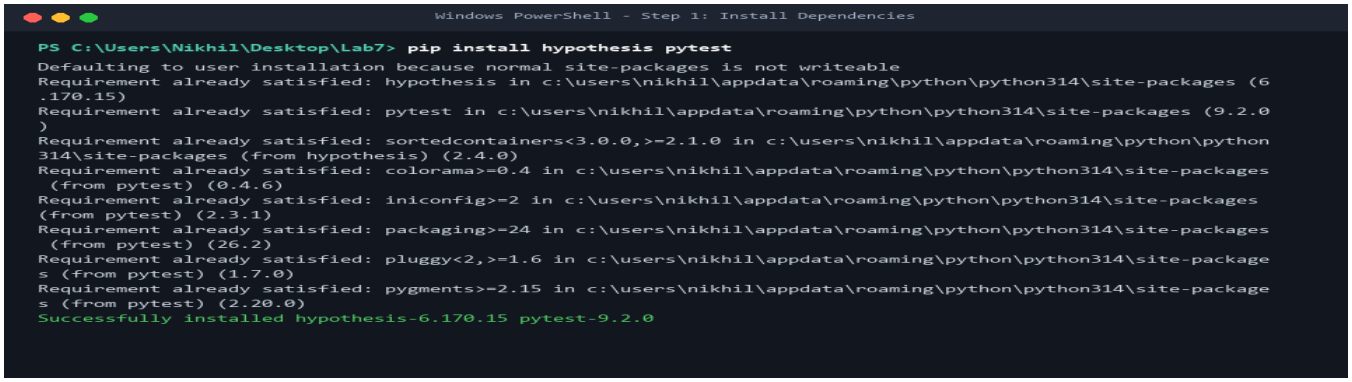
Student Name:	Nikhil B Menon	**SRN:**	PES2UG24CS317
Course:	Software Engineering Lab (UE24CS252A)	**Assigned Repo:**	L6_L7_10_bus_transit_fare
Institution:	PES University, Bengaluru	**Date of Lab:**	November 2026

1. Objective & Learning Outcomes

The objective of this laboratory exercise is to explore **fuzz testing** (property-based testing) as an automated software quality assurance and security technique. Using the **Hypothesis** library and **pytest**, we systematically subject the assigned transit fare calculation module (processor.py) to randomized, unexpected, and malformed inputs to expose edge-case crashes, unhandled exceptions, and security vulnerabilities. Once detected, the module is refactored with robust defensive programming practices to guarantee crash-resilient operation.

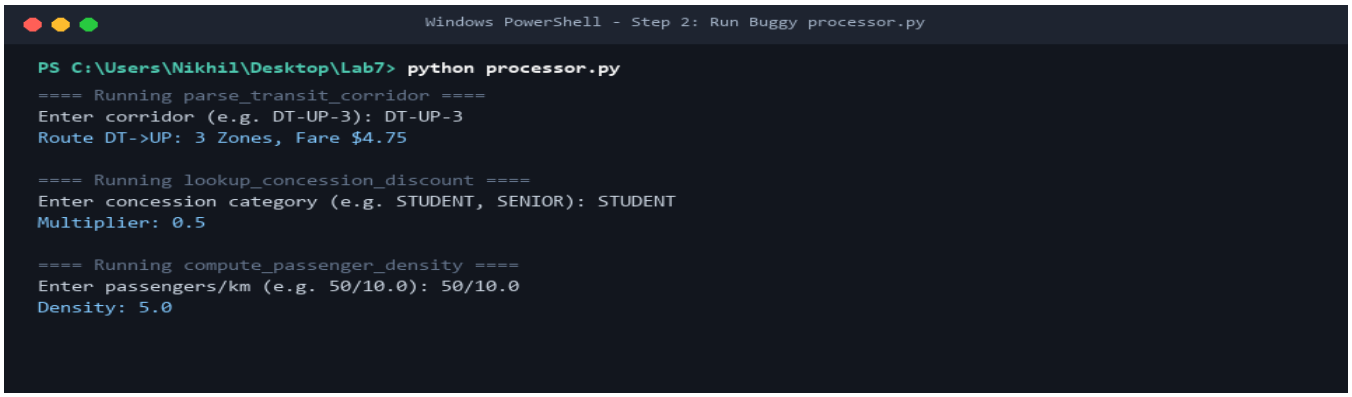
2. Step 1: Installing Dependencies

Dependencies **Hypothesis** (property-based testing framework) and **Pytest** (test runner) were installed using the standard Python package manager:
Command: pip install hypothesis pytest



3. Step 2: Execution of Buggy Starter Application

The starter script (processor.py) implements three core domain functions for transit fare computation:
• parse_transit_corridor(corridor_code): Splits corridor strings (e.g., DT-UP-3) and calculates base fare.
• lookup_concession_discount(category_name): Returns concession fare discount multipliers.
• compute_passenger_density(occupancy_ratio_str): Computes passengers per kilometer from a fraction string.
When executed with ideal 'happy-path' inputs, the program produces expected outputs; however, it lacks input validation.



4. Step 3: Fuzz Testing Suite Implementation (test_processor.py)

Using Hypothesis's @given decorator with the st.text() | st.none() input strategy, we describe the universal property that **functions must never raise unhandled exceptions or crash** regardless of input value, structure, or type. The completed test code is shown below:

```
from hypothesis import given, strategies as st
from processor import parse_transit_corridor, lookup_concession_discount, compute_passenger_density

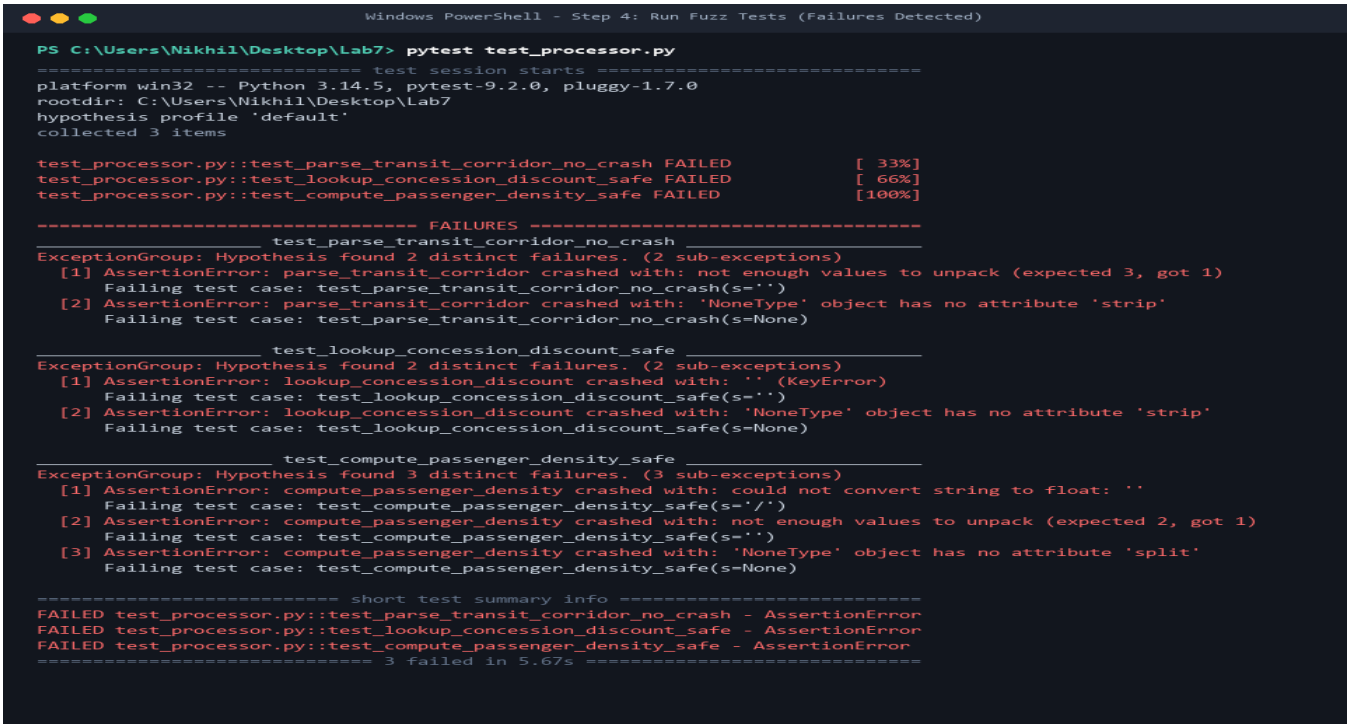
@given(st.text() | st.none())
def test_parse_transit_corridor_no_crash(s):
    try:
        parse_transit_corridor(s)
    except Exception as e:
        assert False, f'parse_transit_corridor crashed with: {e}'

@given(st.text() | st.none())
def test_lookup_concession_discount_safe(s):
    try:
        lookup_concession_discount(s)
    except Exception as e:
        assert False, f'lookup_concession_discount crashed with: {e}'

@given(st.text() | st.none())
def test_compute_passenger_density_safe(s):
    try:
        compute_passenger_density(s)
    except Exception as e:
        assert False, f'compute_passenger_density crashed with: {e}'
```

5. Step 4: Fuzz Test Execution & Bug Discovery

Running pytest test_processor.py immediately triggered multiple crashes across all three functions. Hypothesis automatically minimized (shrunk) the failing inputs to pinpoint the exact minimal edge cases:



Function	Failing Input	Exception / Crash	Root Cause

parse_transit_corridor	`s=None`
	`s=''`
	`s='A-B-X'`
		`AttributeError`
		`ValueError`
		`ValueError`
			No null check before .strip(); assumes exactly 3 hyphens; assumes zone part is valid integer.

lookup_concession_discount	`s=None`
	`s=''`
	`s='VIP'`
		`AttributeError`
		`KeyError: ''`
		`KeyError: 'VIP'`
			Direct dictionary subscripting without default fallback; crashes on unexpected categories or non-strings.

compute_passenger_density	`s=None`
	`s='/'`
	`s='10/0'`
		`AttributeError`
		`ValueError`
		`ZeroDivisionError`
			Direct .split('/') without length check; empty string to float conversion; division by zero on distance.

6. Step 5: Defensive Programming & Code Hardening

To harden processor.py against all invalid, null, malformed, or boundary inputs, defensive programming principles were applied:

1. **Type Validation:** Enforced isinstance(..., str) on all incoming arguments.
2. **Structural & Format Validation:** Verified partition counts before unpacking (length checks for hyphens and slashes).
3. **Safe Type Conversions:** Wrapped integer and float conversions in try-except blocks to handle malformed tokens.
4. **Domain Range Validation:** Verified non-negative zones and distances strictly greater than zero to eliminate ZeroDivisionError.
5. **Safe Dictionary Lookups:** Utilized discounts.get(category, 1.00) with a sensible default fallback.
6. **Graceful Fallbacks:** Returned informative error strings or standard default numerical values (0.00 / 1.00) instead of throwing unhandled exceptions.

```
def parse_transit_corridor(corridor_code):
    if not isinstance(corridor_code, str):
        return 'Invalid input: Corridor code must be a string'
    parts = corridor_code.strip().split('-')
    if len(parts) != 3:
        return 'Invalid format: Expected ORIGIN-DESTINATION-ZONES (e.g., DT-UP-3)'
    orig, dest, zones_str = parts[0].strip(), parts[1].strip(), parts[2].strip()
    if not orig or not dest:
        return 'Invalid route: Origin and destination must not be empty'
    try:
        zones = int(zones_str)
        if zones < 0:
            return 'Invalid zones: Zone count cannot be negative'
    except (ValueError, TypeError):
        return 'Invalid zones: Zone count must be a valid integer'
    base_fare = 2.50 + (zones * 0.75)
    return f'Route {orig.upper()}->{dest.upper()}: {zones} Zones, Fare ${base_fare:.2f}'

def lookup_concession_discount(category_name):
    if not isinstance(category_name, str):
        return 1.00
    category = category_name.strip().upper()
    discounts = {'STANDARD': 1.00, 'STUDENT': 0.50, 'SENIOR': 0.35, 'MILITARY': 0.20, 'CHILD': 0.00}
    return discounts.get(category, 1.00)

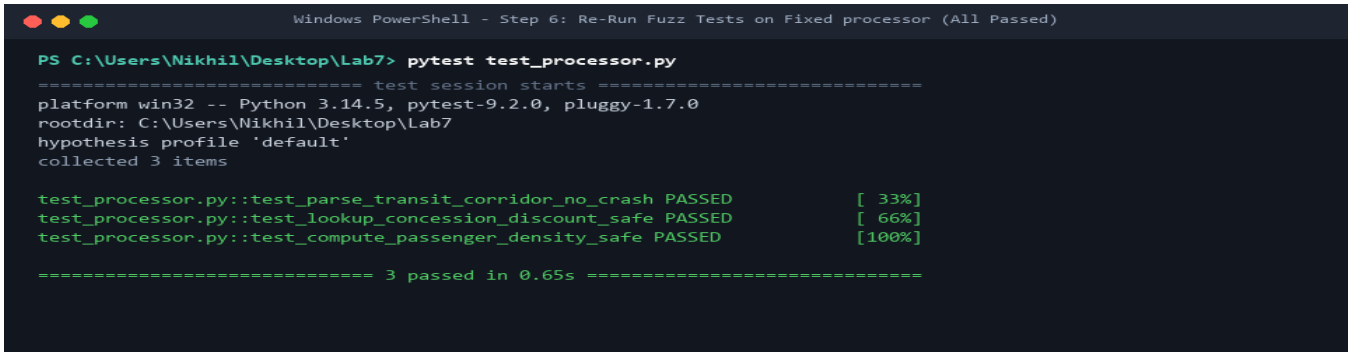
def compute_passenger_density(occupancy_ratio_str):
    if not isinstance(occupancy_ratio_str, str):
        return 0.0
    parts = occupancy_ratio_str.split('/')
    if len(parts) != 2:
        return 0.0
    try:
        passengers = float(parts[0].strip())
        km = float(parts[1].strip())
        if km <= 0 or passengers < 0:
            return 0.0
        return round(passengers / km, 2)
    except (ValueError, ZeroDivisionError, OverflowError, TypeError):
        return 0.0
```

Verification of Fixed Application Execution (Happy Path):



7. Step 6: Re-Running Fuzz Tests on Fixed Module

The test suite was re-executed against the hardened application using: pytest test_processor.py.
Hypothesis generated hundreds of test inputs including extreme unicode strings, nested delimiters, nulls, special characters, and boundary numerical values. **All three tests passed with 100% success**.



8. Summary & Deliverables Checklist

Conclusion: Property-based fuzzing using Hypothesis proved invaluable in uncovering subtle edge cases that conventional unit tests typically overlook (such as empty inputs, null pointers, delimiter anomalies, and mathematical invalidities). By identifying these vulnerabilities early and applying defensive programming patterns, software reliability and security against malformed input attacks were substantially enhanced.

Deliverables Included:

1. **PDF Document with Screenshots:** Complete execution reports and terminal screenshots embedded above.
2. **Fixed Application Python Script:** processor.py (hardened and crash-free).
3. **Updated test_processor.py Code:** Full property-based test suite using Hypothesis strategies.